Ejercicio Final – DOM y Eventos: «TiendaDAW»

Desarrollo Web en Entorno Cliente · 2º DAW · Duración: **2 horas** · Puntuación: **10 puntos**

¿Qué vas a hacer? Una página para llevar los productos de una tienda: darlos de alta, verlos en una tabla, venderlos, reponerlos, borrarlos, buscarlos y ver un resumen.

Archivos:

- `index.html` → la página ya hecha. **No se toca.** Solo la abres para probar.
- `js/clases.js` → la clase `Producto` que te da el profesor. **No se toca**, solo la usas.
- `js/codigo.js` → **aquí escribes TODO tu código**, debajo de los arrays que ya hay.

> **Antes de empezar (5 min):** abre `clases.js` y apunta: qué le pasas al `constructor`, qué getters hay y qué hacen `vender()`, `reponer()`, `getValorTotal()` y `toString()`. Ojo: `vender()` devuelve `true` si ha podido vender y `false` si no hay stock. Abre también `index.html` y apunta los `id`, `name`, clases y `data-` que vas a usar.

1. Que la página arranque — 0,75 p

1. Los `<script>` están en el `<head>`, o sea, se ejecutan **antes** de que exista el HTML. Por eso todo tu código de inicio (listeners, etc.) tiene que ir dentro de `DOMContentLoaded`.
2. `datosIniciales` son objetos normales `{...}`. Recórrelos y por cada uno haz un `new Producto(...)` y mételo en el array `productos`.
3. Al abrir la página solo se ve la sección **Inventario**, con la tabla ya rellena (la tabla es el apartado 4).

2. Menú de arriba — 1 p

Al pulsar un enlace del menú se ve su sección y se ocultan las otras dos.

1. Pon **un solo** `addEventListener("click", …)` en el `<ul id="menu">`, no uno por enlace (eso es la delegación).
2. Cada enlace tiene `data-seccion="alta"`, `"inventario"` o `"resumen"`, que coincide con el `id` de su `<section>`. Léelo con `event.target.dataset.seccion`.
3. Las 3 secciones tienen la clase `seccion`: ocúltalas todas y luego enseña la que toca.
4. El enlace tiene `href="#"`: usa `preventDefault()` para que no salte la página.
5. Si la sección es **resumen**, llama a tu función del resumen (apartado 7).

3. Formulario de alta — 2 p

a) Código automático. Cuando cambies la categoría del select (evento `change`), se rellena el input `codigo` así:

- 3 primeras letras de la categoría en mayúscula + `-` + número con 3 cifras.
- El número es: cuántos productos hay ya de esa categoría + 1. Ej.: hay 2 de Electrónica → `ELE-003`.
- Si vuelves a «-- Elige categoría --» (value vacío), el código se queda vacío.

Cosas que te sirven:

```
"Deporte".slice(0, 3).toUpperCase()   // "DEP"
String(7).padStart(3, "0")            // "007"  (rellena con ceros por delante
hasta 3)
productos.filter(p => condición).length   // cuántos cumplen la condición
```

b) Botón «Añadir producto». Es un `type="submit"`, así que escucha el evento `submit` del **formulario** (no el click del botón) y pon `preventDefault()` para que no se recargue la página.

Comprueba esto y, si falla algo, enseña **todos los errores juntos** en una lista dentro de `#mensajes` (como en el EJ14):

- Categoría elegida.
- Nombre de 3 letras o más (quitando espacios con `trim()`) y que **no exista ya** otro producto con ese nombre (da igual mayúsculas o minúsculas → compara los dos en `toLowerCase()`). Pista: `some()`.
- Precio: número mayor que 0 (puede tener decimales → `parseFloat`).
- Stock: no vacío, número entero y 0 o más. Pista: `Number.isInteger(numero)` devuelve `true` si no tiene decimales.

El nombre se guarda con la 1ª letra en mayúscula y el resto en minúscula: `"tECLADO"` → `"Teclado"`.

Cosas que te sirven:

```
texto.charAt(0)    // primera letra
texto.slice(1)     // todo menos la primera letra
```

Para que los errores salgan en rojo, mete la lista dentro de un `<div class="alert alert-danger">`. Para el mensaje de éxito, `alert alert-success`.

Si todo está bien: `new Producto(...)`, `push` al array, mensaje verde «Producto ELE-003 añadido», vaciar el formulario (`reset()`) y volver a pintar la tabla.

4. Pintar la tabla — 2 p

Haz una función `pintarTabla()` que vacíe `#cuerpoTabla` y meta una fila `<tr>` por cada producto con: código, nombre, categoría, precio, stock y 3 botones (**Vender**, **Reponer +5**, **Eliminar**).

Cómo meter la fila (template literals): en vez de comillas normales usa **comillas invertidas** `` ` `` (tecla a la derecha de la P). Dentro puedes meter variables con `${ }` y escribir en varias líneas:

```
const nombre = "Ana";
div.innerHTML += `<p class="x">Hola ${nombre}, tienes ${3 + 2} mensajes</p>`;
// es lo mismo que:  "<p class='x'>Hola " + nombre + ", tienes " + (3 + 2) + "
mensajes</p>"
```

Recuerda: antes del bucle `innerHTML = ""` y dentro del bucle `+=`.

- Precio con 2 decimales y el símbolo: `29.99 €` → `toFixed(2)`.
- Cada botón lleva **dos** atributos `data-`: qué hace y de qué producto es. Ej.: `data-accion="vender"` `data-codigo="ELE-001"`. Lo mismo con `reponer` y `eliminar`.

- Color de la fila (clase en el `<tr>`): stock 0 → `table-danger` (rojo). Stock 1 o 2 → `table-warning` (amarillo). Si no, sin clase.
- Si no hay productos que enseñar (por ejemplo, porque el buscador no encuentra nada), pon una sola fila que diga «No hay productos». Pista: `<td colspan="6">` ocupa las 6 columnas.

5. Botones de la tabla — 2 p

Igual que en el menú: **un solo** listener `click` en `#cuerpoTabla`, no uno por botón. Con `event.target.dataset.accion` sabes qué botón es y con `event.target.dataset.codigo` de qué producto. Si has pulsado en otro sitio (una celda), `dataset.accion` será `undefined` → no hagas nada.

- **Vender:** busca el producto (`find`) y llama a su `vender(1)`. Si devuelve `false` → `alert("No hay stock de " + nombre)`. Si devuelve `true` → añade al array `ventas` un texto como `"19:24:45 - 1 x Teclado - 45.50 €"`.
  Para la hora: `new Date().toLocaleTimeString()` te da `"19:24:45"`.
- **Reponer +5:** busca el producto y llama a su `reponer(5)`.
- **Eliminar:** pregunta con `confirm()` enseñando el `toString()` del producto. Si dice que sí, busca su **posición** (`findIndex`) y quítalo con `splice`.
- Después de cualquier botón, vuelve a pintar la tabla.

6. Buscar, filtrar y ordenar — 1,25 p

1. **Buscador** (`#buscador`): con el evento `input` (salta con cada letra que escribes) se ven solo los productos cuyo nombre contiene lo escrito. Da igual mayúsculas o minúsculas → pasa los dos a `toLowerCase()` y usa `includes()`.
2. Si en el buscador pulsas la tecla **Escape** (evento `keydown`, `event.key === "Escape"`), se vacía y se ven todos.
3. **Select de categoría** (`#filtroCategoria`, evento `change`): solo esa categoría; con «Todas» se ven todas.
4. Los dos filtros funcionan **a la vez** (ej.: categoría Electrónica + texto «rat»). Truco: que `pintarTabla()` lea siempre el buscador y el select y filtre con `filter()` antes de pintar. Así, si vendes o borras algo, los filtros se mantienen.
5. **Botón ordenar** (`#btnOrdenar`): cada vez que lo pulsas cambia entre precio de menor a mayor y de mayor a menor, y su texto cambia entre `Ordenar por precio ↑` y `Ordenar por precio ↓`.

> **Ojo con sort:** `sort()` cambia el array que ordenas. Ordena el resultado del `filter` (que ya es un array nuevo), no `productos` directamente.
>
> ```
> lista.sort((a, b) => a.getPrecio() - b.getPrecio())   // menor a mayor
> lista.sort((a, b) => b.getPrecio() - a.getPrecio())   // mayor a menor
> ```
>
> Para saber en qué orden toca, guarda una variable fuera de las funciones (ej. `let ordenAscendente = true;`) y cámbiala al pulsar.

7. Resumen — 1 p

Dentro de `#datosResumen` enseña:

- Cuántos productos hay.
- Cuántas unidades hay en total (suma de todos los stocks).
- Valor del inventario: suma de `getValorTotal()` de todos, con 2 decimales.
- Nombre del producto más caro.
- Nombres de los agotados (stock 0) separados por comas, o «Ninguno». Pista: `array.join(", ")` junta un array en un texto.

Dentro de `#historial`: un `<li>` por cada texto del array `ventas`, o «Sin ventas» si está vacío.

> **Extra A (+0,5 p, solo si sobra tiempo):** en `codigo.js` crea la clase `ProductoOferta extends Producto`, con un atributo privado `#descuento` (ej. 20 = 20 %) y un método `getPrecioFinal()` que devuelva el precio con el descuento aplicado. Crea uno al arrancar y en la tabla enseña su precio final y la palabra OFERTA. Para saber si un producto es de oferta: `p instanceof ProductoOferta` (da `true` o `false`).

> **Extra B (no puntúa, es del tema siguiente):** en vez de `datosIniciales`, carga los productos desde `productos.json` con `fetch` (mira el apartado 14 de la chuleta). Necesita Live Server.

Orden recomendado y tiempo

Minutos	Qué hacer
0 – 10	Leer el enunciado, la clase y el HTML. Apartado 1.
10 – 25	Apartado 2 (menú).
25 – 50	Apartado 4 (tabla). Hazlo **antes** que el 3 para ir viendo los productos que añades.
50 – 75	Apartado 3 (formulario).
75 – 95	Apartado 5 (botones de la tabla).
95 – 110	Apartados 6 y 7.
110 – 120	Probar todo con la consola abierta (F12). Extras si sobra tiempo.